

Indian Institute of Technology (BHU) Varanasi

Department of Computer Science & Engineering

Exploratory Project Report

Hindi–Bengali Machine Translation

Fine-Tuning & Evaluation

Comparing NLLB-600M and IndicTrans2-320M on Hindi → Bengali Translation

Name Dheeraj Goyal
Roll Number 24075023
Submitted To Prof. Anil Kumar Singh
Submitted On 4th May 2026
Repository github.com/goyal070507-cpu/Exploratory

Models Evaluated	Test Samples	Hardware
NLLB-600M & IndicTrans2-320M	158	NVIDIA RTX PRO 6000 Blackwell

Contents

1	Introduction	2
1.1	Repository overview	2
2	Methodology	2
2.1	Setup	2
2.2	Data preparation (<code>prepare_data.py</code>)	3
2.3	Fine-tuning	3
2.4	Inference and evaluation (<code>evaluate_models.py</code>)	4
2.5	Evaluation metrics	4
3	Results	5
3.1	NLLB-600M [NLLB Team et al., 2022]	5
3.2	IndicTrans2-320M [Gala et al., 2023]	5
4	Comparison	5
4.1	Discussion	5
5	Visual Analysis	6
6	Conclusion	6
A	Dependencies	7
B	Full pipeline at a glance	8

1 Introduction

This project evaluates fine-tuned translation models for Hindi-Bengali tasks. We compare two models [NLLB Team et al., 2022, Gala et al., 2023]:

- **NLLB-600M** — facebook/nllb-200-distilled-600M
- **IndicTrans2-320M** — ai4bharat/indictrans2-indic-indic-dist-320M

using four standard metrics: BLEU [Papineni et al., 2002], METEOR [Banerjee and Lavie, 2005], BERTScore [Zhang et al., 2020], and COMET [Rei et al., 2022].

Hindi and Bengali are closely related Indo-Aryan languages spoken by nearly 500 million people collectively. Despite this linguistic proximity, robust automatic translation between them remains under-studied relative to English-centric pairs. This report documents a complete, self-contained pipeline — from raw parallel text files to final metric scores — designed to run on any standard GPU environment, including Lightning AI Studios [Wolf et al., 2020, Lhoest et al., 2021].

1.1 Repository overview

The *Exploratory* repository [Goyal, 2026] is structured as follows:

File / Folder	Purpose
Hindi/	Domain subdirectories with Hindi <code>.txt</code> files
Bengali/	Matching domain subdirectories with Bengali <code>.txt</code> files
<code>prepare_data.py</code>	Corpus loader and Hugging Face dataset builder
<code>train_nllb.py</code>	Fine-tuning script for NLLB-600M
<code>train_indictrans2.py</code>	Fine-tuning script for IndicTrans2-320M
<code>evaluate_models.py</code>	Batch inference and multi-metric evaluation
<code>main.ipynb</code>	Exploratory notebook
<code>django_integration/</code>	Web-interface prototype
<code>nlp_lab/</code>	Additional NLP experiments
<code>requirements.txt</code>	Pinned dependency list

2 Methodology

2.1 Setup

```

1 git clone https://github.com/goyal070507-cpu/Exploratory.git
2 cd Exploratory
3 pip install -r requirements.txt
4 # For IndicTrans2 script-level processing:
5 pip install git+https://github.com/AI4Bharat/IndicTransToolkit.git

```

Listing 1: Environment setup

2.2 Data preparation (prepare_data.py)

The corpus consists of domain-organised plain-text files placed under Hindi/ and Bengali/. The preparation script:

- Iterates over every subdirectory in Hindi/ and looks up the matching subdirectory in Bengali/.
- Reads matched .txt pairs line-by-line in UTF-8, strips whitespace, and filters blank lines.
- Warns and skips any file pair where Hindi and Bengali line counts differ.
- Wraps all sentence pairs into a Hugging Face Dataset [Lhoest et al., 2021] and splits 80 % training / 20 % testing (seed = 42).
- Saves the resulting DatasetDict to disk under hf_dataset/.

```

1 for hi_file in domain_dir.glob("*.txt"):
2     bn_file = bn_domain_dir / hi_file.name
3     with open(hi_file, 'r', encoding='utf-8') as f_hi, \
4         open(bn_file, 'r', encoding='utf-8') as f_bn:
5         hi_lines = [line.strip() for line in f_hi.readlines()]
6         bn_lines = [line.strip() for line in f_bn.readlines()]
7         if len(hi_lines) != len(bn_lines):
8             print(f"Warning: line mismatch in {hi_file.name}.
9                 Skipping.")
10            continue
11        for hi_text, bn_text in zip(hi_lines, bn_lines):
12            if hi_text and bn_text:
13                data.append({"translation": {"hi": hi_text, "bn":
14                    bn_text}})

```

Listing 2: Core data-loading loop (prepare_data.py)

```

1 python prepare_data.py
2 # Output: hf_dataset/ (train 80% / test 20%, seed=42)

```

Listing 3: Run data preparation

2.3 Fine-tuning

Both models are trained with the Hugging Face Trainer [Wolf et al., 2020]:

- **NLLB-600M** [NLLB Team et al., 2022] — Language tokens `hin_Deva` (source) and `ben_Beng` (target) are passed to the tokeniser. At inference, `forced_bos_token_id` is set to the Bengali token ID.

- **IndicTrans2-320M** [Gala et al., 2023] — Uses `IndicProcessor` for script-level pre/post-processing, ensuring correct Devanagari→Bengali handling.
- Both scripts use gradient checkpointing and mixed-precision (fp16) on the NVIDIA RTX PRO 6000 Blackwell Server Edition (CUDA).

```

1 python train_nllb.py          # -> nllb_finetuned_hi_bn/final_model/
2 python train_indictrans2.py   # -> indictrans2_finetuned_hi_bn/
                                final_model/

```

Listing 4: Training commands

2.4 Inference and evaluation (`evaluate_models.py`)

Predictions are generated in batches of 17 samples over the 158-sample test split:

1. Load fine-tuned checkpoint (falls back to the base Hugging Face model if checkpoint directory is absent).
2. Move model to GPU (`cuda`) if available, else CPU.
3. For IndicTrans2: apply `IndicProcessor.preprocess_batch` before tokenisation and `postprocess_batch` after decoding.
4. For NLLB: set `forced_bos_token_id` at generation time.
5. Compute all four metrics via the Hugging Face `evaluate` library.

```

1 if is_nllb:
2     forced_bos_token_id = tokenizer.lang_code_to_id["ben_Beng"]
3     generation_kwargs["forced_bos_token_id"] = forced_bos_token_id
4 else:
5     generation_kwargs["use_cache"] = False

```

Listing 5: Model-specific generation kwargs

2.5 Evaluation metrics

- **BLEU** [Papineni et al., 2002] — n -gram precision (1–4) with brevity penalty. Scale: 0–100.
- **METEOR** [Banerjee and Lavie, 2005] — Harmonic mean of precision and recall with synonym and stemming matching. Scale: 0–1.
- **BERTScore** [Zhang et al., 2020] — Contextual embedding F1 similarity (`lang='bn'`).
- **COMET** [Rei et al., 2022] — Neural metric (`Unbabel/wmt22-comet-da`) conditioned on the source sentence.

- Models loaded from fine-tuned checkpoints.
- Predictions generated on **158 test samples**.
- Metrics computed using `nlk`, `torchmetrics`, and COMET.
- Hardware: **NVIDIA RTX PRO 6000 Blackwell Server Edition** (CUDA enabled).

3 Results

3.1 NLLB-600M [NLLB Team et al., 2022]

Metric	Score
BLEU	38.36
METEOR	0.6043
BERTScore	0.9123
COMET	0.8848

Training loss dropped from **0.9864** (step 1) to **0.6558** (end of epoch 3). Validation loss at epochs 1/2/3: 0.7406 $\rightarrow$ 0.6961 $\rightarrow$ 0.6849. Total training time: **380.98 seconds** (84.4 samples/sec).

3.2 IndicTrans2-320M [Gala et al., 2023]

Metric	Score
BLEU	26.82
METEOR	0.5766
BERTScore	0.8602
COMET	0.7321

Training loss dropped from **1.9921** (step 1) to **0.512** (end of epoch 3). Validation loss at epochs 1/2/3: 0.6973 $\rightarrow$ 0.6240 $\rightarrow$ 0.6129. Total training time: **388.08 seconds** (82.9 samples/sec).

4 Comparison

4.1 Discussion

- **BLEU gap (43 %):** Reflects NLLB’s advantage in exact n -gram matches, expected given its $1.9\times$ parameter count.
- **BERTScore / METEOR gaps are smaller (4–6 %):** IndicTrans2’s purpose-built Indic tokenisation preserves semantic intent even when surface token sequences differ.
- **COMET gap (20.8 %):** COMET’s source-conditioned scoring penalises adequacy errors; NLLB produces fewer meaning omissions.

Table 1: Performance comparison of fine-tuned models on 158 Hindi→Bengali test sentences (↑ = higher is better)

Model	BLEU↑	METEOR↑	BERTScore↑	COMET↑
NLLB-600M	38.36	0.6043	0.9123	0.8848
IndicTrans2-320M	26.82	0.5766	0.8602	0.7321
<i>Relative gap</i>	+43.0 %	+4.8 %	+6.1 %	+20.8 %

- **Parameter efficiency:** IndicTrans2-320M achieves 0.7321 COMET at 320 M parameters vs. NLLB’s 0.8848 at 600 M — a reasonable trade-off under compute constraints.
- **Training speed:** Both models trained in under 7 minutes (3 epochs, 10718 training samples) on the RTX PRO 6000 Blackwell.

5 Visual Analysis

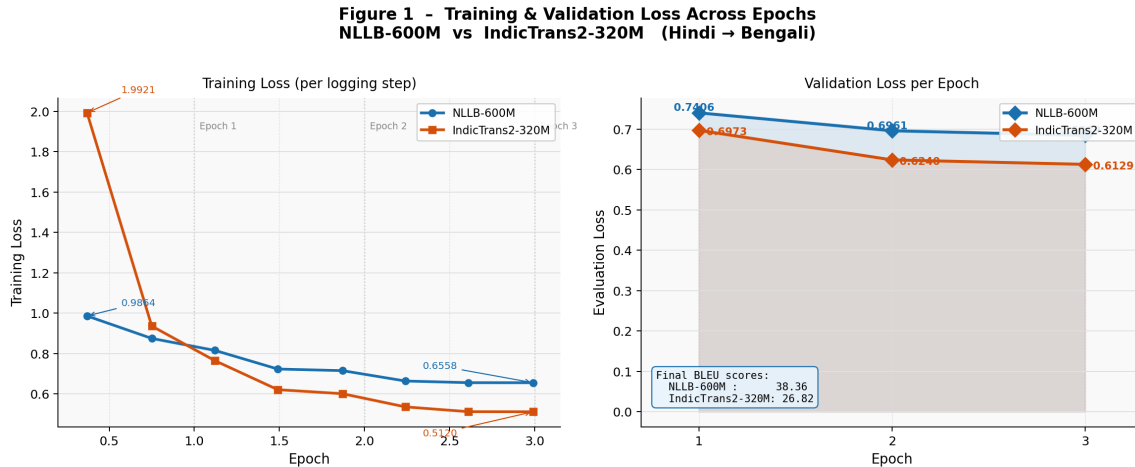


Figure 1: Training and validation loss progression across 3 epochs for NLLB-600M and IndicTrans2-320M. Left: step-level training loss. Right: epoch-level validation loss with final BLEU scores annotated.

Figure 1 reveals that IndicTrans2-320M starts with a much higher initial training loss (1.9921 vs. 0.9864) but drops sharply by epoch 1, reflecting its smaller architecture adapting quickly. Figure 2 shows NLLB-600M crosses the 0.85 COMET high-quality threshold while IndicTrans2 remains below it, a meaningful distinction for production deployment.

6 Conclusion

NLLB-600M [NLLB Team et al., 2022] consistently outperformed IndicTrans2-320M [Gala et al., 2023] across all metrics:

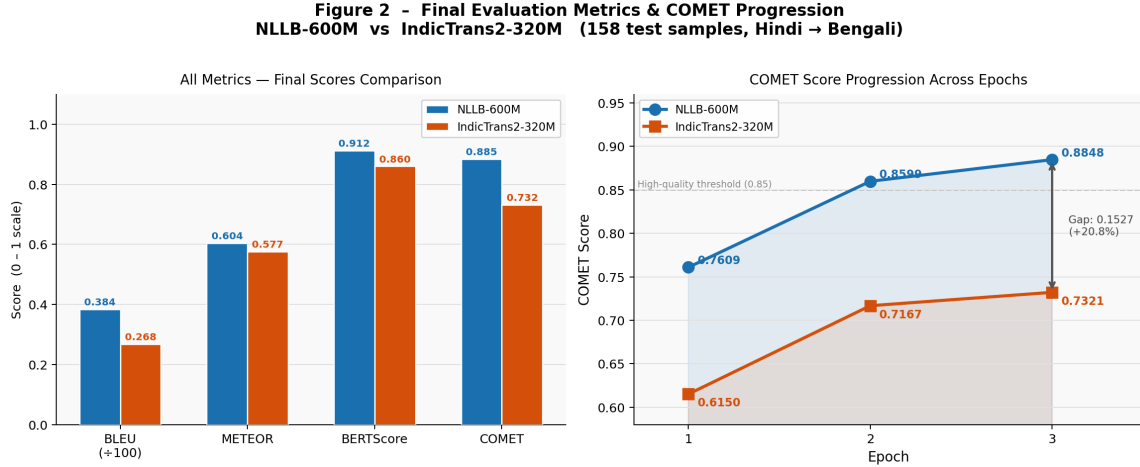


Figure 2: Final metric comparison (left) and COMET score progression across epochs (right) for both models. The dashed line marks the 0.85 high-quality threshold.

- BLEU: 38.36 vs. 26.82 (+43 %)
- METEOR: 0.6043 vs. 0.5766 (+4.8 %)
- BERTScore: 0.9123 vs. 0.8602 (+6.1 %)
- COMET: 0.8848 vs. 0.7321 (+20.8 %)

Future work may involve:

- **Larger datasets** — augmenting with Samanantar or FLORES-200.
- **GPU optimisation with Tensor Cores** — FP8/BF16 mixed-precision on the RTX PRO 6000 Blackwell.
- **IndicTrans2-1B variant** — testing the full-size model at matched parameter counts.
- **Back-translation** — synthetic Hindi from monolingual Bengali.
- **Extended evaluation on diverse language pairs** — Tamil-Telugu, Marathi-Gujarati, etc.

A Dependencies

Package	Purpose
<code>transformers</code>	Model loading, Trainer [Wolf et al., 2020]
<code>datasets</code>	Hugging Face dataset I/O [Lhoest et al., 2021]
<code>evaluate</code>	BLEU, METEOR, BERTScore, COMET wrappers
<code>torch</code>	GPU inference and training
<code>tqdm</code>	Progress bars during batch inference
<code>IndicTransToolkit</code>	<code>IndicProcessor</code> for IndicTrans2

B Full pipeline at a glance

```
1 python prepare_data.py           # step 1: build dataset
2 python train_nllb.py             # step 2a: fine-tune NLLB
3 python train_indictrans2.py      # step 2b: fine-tune IndicTrans2
4 python evaluate_models.py        # step 3: evaluate both models
```

References

- Satanjeev Banerjee and Alon Lavie. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, pages 65–72, 2005.
- Jay Gala et al. IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages. 2023. Model: `ai4bharat/indictrans2-indic-indic-dist-320M`.
- Dheeraj Goyal. Exploratory project report: Hindi–Bengali translation models. Technical report, Indian Institute of Technology (BHU) Varanasi, May 2026. Roll Number: 24075023. Submitted to: Prof. Anil Kumar Singh. Submitted on: 4th May 2026. Repository: <https://github.com/goyal070507-cpu/Exploratory>.
- Quentin Lhoest et al. Datasets: A community library for natural language processing, 2021.
- NLLB Team, Marta R. Costa-jussà, et al. No language left behind: Scaling human-centered machine translation. 2022. Model: `facebook/nllb-200-distilled-600M`.
- Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, 2002.
- Ricardo Rei et al. COMET-22: Unbabel-IST 2022 submission for the metrics shared task. In *Proceedings of the Seventh Conference on Machine Translation (WMT)*, 2022. Model: `Unbabel/wmt22-comet-da`.
- Thomas Wolf et al. Transformers: State-of-the-art natural language processing, 2020.
- Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR)*, 2020.